

Recitation 7 RTOS Scheduling

Agenda

1. Why RTOS? Picking up from last recitation
 2. Core RTOS concepts
 3. How Mbed schedules threads
 4. Mbed Thread API
 5. EventQueue
 6. Demo 1 Two LEDs, one thread (broken) vs two threads (fixed)
 7. Demo 2 LSM6DSL sensor read thread + print thread
-

From Last Time

Last recitation we ran into a real problem. When the LSM6DSL fired at 104Hz, reading I2C and calling printf in the same loop meant the two operations were blocking each other. The fix was to split them into two threads and use an EventQueue to pass data between them. That worked, but we kind of just did it without asking why it worked or what was actually managing those two threads underneath.

That is exactly what this recitation is about. The answer is Mbed RTOS, and understanding it properly will make you a much better embedded developer.

What is an RTOS?

A regular operating system like Linux or Windows is designed around fairness it tries to give every application a reasonable share of CPU time and generally does not make promises about when exactly something will run.

A real-time operating system is designed around deadlines. It does not primarily care about fairness. It cares about making sure that when a task needs to run, it runs on time. In embedded systems this matters a lot. If you are reading a sensor at 104Hz, a 200ms delay in the read is not just slow it is wrong.

Mbed OS is an RTOS. It gives you tools to define multiple tasks (threads), assign priorities to them, and let the kernel figure out which one runs at any given moment.

Core Concepts

Tasks and Threads

In Mbed, the unit of work that the scheduler manages is called a thread. Each thread is basically an independent function that runs as if it has the CPU to itself. Behind the scenes the RTOS is rapidly switching between threads to give the illusion of parallelism, but on our STM32 there is only one core only one thread is actually executing at any instant.

Task States

At any point in time, a thread is in one of four states:

Running the thread currently has the CPU and is executing instructions. Only one thread can be in this state at a time.

Ready the thread could run, it has nothing blocking it, but the CPU is currently being used by a higher priority thread. It is waiting its turn.

Waiting the thread cannot run right now even if the CPU were free. This happens when a thread calls `ThisThread::sleep_for()`, waits on a semaphore, or waits for data. It is not wasting CPU cycles while waiting the scheduler simply does not schedule it until whatever it is waiting on resolves.

Inactive the thread has not been started yet, or it has already finished and terminated. Inactive threads consume no system resources.

This distinction between ready and waiting is important. A thread that calls `sleep_for(500ms)` is waiting it is not spinning in a loop burning CPU. The scheduler parks it and moves on to other threads.

Context Switch

When the RTOS decides to switch from one thread to another, it has to save the state of the currently running thread so it can resume it later. This includes all the CPU registers, the program counter, the stack pointer everything that defines where that thread is and what it was doing. This saved state is called the thread's context, and the act of saving one thread's context and loading another's is called a context switch.

Each thread gets its own private stack in memory for exactly this reason. When a thread is switched out, its stack holds its local variables and its saved registers. When it is switched back in, everything is restored and it continues as if nothing happened.

How Mbed Schedules Threads

This is a question that comes up a lot because lecture covers EDF and RMS as theoretical scheduling algorithms, and then you come to Mbed and it does not use either of them.

Mbed RTOS uses **preemptive, priority-based scheduling with round-robin for equal-priority threads**.

Here is what that means in practice:

Priority-based every thread is assigned a priority when it is created. The scheduler always picks the highest-priority thread that is in the ready state. A lower-priority thread simply does not run as long as a higher-priority thread is ready.

Preemptive if a high-priority thread becomes ready while a lower-priority thread is running, the lower-priority thread is immediately paused and the high-priority thread takes over. The lower-priority thread does not get to finish its current work first. This is the key difference from a cooperative scheduler where threads voluntarily yield.

Round-robin for equal priority if two threads have the same priority and both are ready, Mbed gives each one a time slice (1ms by default) and alternates between them.

This is not EDF or RMS. EDF and RMS are algorithms for assigning priorities optimally to guarantee deadlines are met. Mbed gives you the mechanism preemptive priority scheduling and it is your job as the developer to assign priorities sensibly. If you assign priorities correctly (higher priority to more time-critical tasks), Mbed will honor them.

Priority Levels in Mbed

Mbed defines priority as an enum. The main ones you will use:

```
osPriorityLow
osPriorityNormal    // default if you don't specify anything
osPriorityAboveNormal
osPriorityHigh
osPriorityRealtime
```

The main thread (your `int main()`) runs at `osPriorityNormal` by default. This means any thread you create with `osPriorityHigh` or above will preempt main as soon as it is started and becomes ready. Keep this in mind when doing initialization work in main if you start a high priority thread before you are done setting up, it will immediately take over.

Mbed Thread API

Creating a Thread

```
#include "mbed.h"

void my_task() {
    while (true) {
        // do something
        ThisThread::sleep_for(500ms);
    }
}

int main() {
    Thread t;
    t.start(my_task);

    while (true) {
        ThisThread::sleep_for(1s);
    }
}
```

`Thread t` creates a thread object. `t.start(my_task)` launches it. The thread function runs concurrently with main from that point on.

Setting Priority

```
Thread t(osPriorityHigh);  
t.start(my_task);
```

You pass the priority as a constructor argument. You can also set it after creation with `t.set_priority()`.

Sleeping

```
ThisThread::sleep_for(200ms); // sleep for 200 milliseconds  
ThisThread::sleep_for(1s);    // sleep for 1 second
```

`ThisThread::sleep_for()` blocks the calling thread for the specified duration. While it is sleeping it is in the blocked state and the scheduler runs other threads. This is very different from a busy wait loop it does not consume CPU.

Joining

```
t.join(); // wait for thread t to finish before continuing
```

Not always needed if your threads run forever, but useful when you want the main thread to wait for a worker thread to complete.

EventQueue

What it is and why it exists

Even with threads, there are still situations where you cannot just call a function directly. The most common case is inside an ISR. As covered last recitation, you cannot call `printf` inside an ISR, you cannot do I2C reads, and generally you cannot call anything that might block or use RTOS primitives. The ISR has to finish fast.

The EventQueue solves this by letting you defer a function call to a different context. Instead of calling a function directly, you post it to a queue. A separate thread picks it up and runs it when it gets CPU time. The ISR stays tiny, and the heavy work happens safely in a normal thread context.

This is exactly the pattern we used at the end of recitation 6 with the LSM6DSL. The ISR set a flag, the acquisition thread read the sensor, and then posted a print job to the EventQueue. The print thread ran the job when it was scheduled. Now you know why that works and what is underneath it.

Creating and starting an EventQueue

```
#include "mbed.h"
```

```
EventQueue queue(32 * EVENTS_EVENT_SIZE);
Thread event_thread;

int main() {
    event_thread.start(callback(&queue, &EventQueue::dispatch_forever));
}
```

You create an EventQueue with a fixed memory size the argument specifies how much space to pre-allocate for pending events. Then you start a thread that runs `dispatch_forever()`, which sits in a loop pulling events off the queue and executing them one by one. That thread becomes your event processing context.

The reason EventQueue uses a fixed pre-allocated memory block is that allocating from the heap is not ISR-safe. By reserving memory up front at creation time, posting an event from an ISR is safe and will never block.

Posting events with queue.call()

```
#include "mbed.h"

EventQueue queue(32 * EVENTS_EVENT_SIZE);
Thread event_thread;

void print_result(float value) {
    printf("value: %.3f\r\n", value);
}

void some_isr() {
    queue.call(print_result, 3.14f); // safe to call from ISR
}

int main() {
    event_thread.start(callback(&queue, &EventQueue::dispatch_forever));
}
```

`queue.call()` schedules a function to run in the context of the thread running `dispatch_forever()`. It returns immediately the function does not run yet, it just gets added to the queue. The dispatch thread picks it up and runs it whenever it next gets CPU time.

You can pass arguments directly to `queue.call()` as shown above. This is how you move data from an ISR into a safe context without shared globals.

One important thing about EventQueue priority

The EventQueue itself has no concept of event priority. If multiple events are queued up, they run in the order they were posted (time-based). If you need different priorities for different kinds of events, you create separate EventQueue instances and run each one on a thread with the appropriate priority level. The priority

of the dispatch thread is what determines how urgently the events in that queue get processed relative to other threads in the system.

queue.call vs queue.event

There are two common ways to post to a queue. `queue.call(func, args...)` posts a one-shot call immediately. `queue.event(func)` returns a wrapper that you can attach to things like `InterruptIn::rise()` or `Ticker::attach()`, so that whenever that interrupt fires, the function is automatically posted to the queue instead of running in ISR context.

```
InterruptIn button(SW2);

void on_press() {
    printf("button pressed\r\n");
}

int main() {
    event_thread.start(callback(&queue, &EventQueue::dispatch_forever));
    button.rise(queue.event(on_press)); // on_press runs in event thread,
not ISR
}
```

This pattern keeps ISR context clean and all the real work happens safely in user context.

Demo 1 Two LEDs

The Problem with a Single Thread

Here is the code you tested on the board:

```
#include "mbed.h"

DigitalOut led1(LED1);
DigitalOut led2(LED2);

int main() {
    while (true) {
        led1 = !led1;
        ThisThread::sleep_for(500ms);
        led2 = !led2;
        ThisThread::sleep_for(500ms);
    }
}
```

Try to make LED1 blink at 200ms and LED2 blink at 1000ms in this code. You cannot do it. The two LEDs are completely coupled whatever you do to one affects the timing of the other because they share the same sequential loop. LED2 cannot toggle until LED1 has finished its sleep, and vice versa.

This is the fundamental limitation of single-threaded programming for concurrent tasks. Tasks that need to run independently at different rates cannot share a single execution thread without interfering with each other.

The Fix Two Threads

```
#include "mbed.h"

DigitalOut led1(LED1);
DigitalOut led2(LED2);

void led1_task() {
    while (true) {
        led1 = !led1;
        ThisThread::sleep_for(200ms);
    }
}

void led2_task() {
    while (true) {
        led2 = !led2;
        ThisThread::sleep_for(1000ms);
    }
}

int main() {
    Thread t1;
    Thread t2;

    t1.start(led1_task);
    t2.start(led2_task);

    while (true) {
        ThisThread::sleep_for(1s);
    }
}
```

Now LED1 blinks at 200ms and LED2 blinks at 1000ms independently. Each thread has its own sleep, its own state, its own stack. The RTOS switches between them fast enough that from our perspective they appear to be running at the same time.

What is actually happening: when `led1_task` calls `sleep_for(200ms)` it goes into the blocked state. The scheduler immediately switches to `led2_task`. When that also sleeps, the scheduler switches to main. When the 200ms is up, `led1_task` moves back to ready and the scheduler picks it up again. This happens continuously and very fast.

Adding Priority

```
Thread t1(osPriorityHigh);           // LED1 is more important
Thread t2(osPriorityNormal);         // LED2 is less important

t1.start(led1_task);
t2.start(led2_task);
```

With this setup, whenever `led1_task` is in the ready state, it will preempt `led2_task` if `led2_task` happens to be running. For simple LED blink tasks where both threads spend most of their time sleeping this does not change much visually, but the mechanism is there and it matters when tasks are doing real work.

Demo 2 LSM6DSL with Threads

This brings everything together. We have a sensor that produces data at 104Hz. We want to read it reliably and also print the data to serial. These two operations should not block each other.

The setup is the same as last recitation the LSM6DSL on I2C, INT1 pin on PD_11 for the data-ready interrupt.

Thread Design

We use two threads:

Acquisition thread (high priority) waits for the data-ready interrupt flag, reads the sensor over I2C immediately when data is available. This is time-critical. If we miss a sample window at 104Hz we lose data.

Print thread (normal priority) takes the most recent sensor values and prints them. Printing is slow and not time-critical. It should not get in the way of reading.

Data is passed between threads using a shared struct. In this recitation we are not covering mutexes yet (that is next week), so we keep the design simple: the acquisition thread writes the struct, the print thread reads it. For now this is fine because the two fields being written are small and the risk of a torn read is low on a 32-bit processor. Next recitation we will protect shared data properly.

```
#include "mbed.h"

I2C i2c(PB_11, PB_10);

#define LSM6DSL_ADDR    (0x6A << 1)
#define WHO_AM_I        0x0F
#define CTRL1_XL         0x10
#define CTRL2_G          0x11
#define CTRL3_C          0x12
#define DRDY_PULSE_CFG  0x0B
#define INT1_CTRL        0x0D
#define STATUS_REG       0x1E
#define OUTX_L_G         0x22
#define OUTX_L_XL        0x28

InterruptIn int1_pin(PD_11, PullDown);
volatile bool data_ready = false;
```



```

void data_ready_isr() {
    data_ready = true;
}

struct SensorData {
    float acc[3];
    float gyro[3];
};

volatile SensorData latest;

bool write_reg(uint8_t reg, uint8_t val) {
    char buf[2] = {(char)reg, (char)val};
    return (i2c.write(LSM6DSL_ADDR, buf, 2) == 0);
}

bool read_reg(uint8_t reg, uint8_t &val) {
    char r = (char)reg;
    if (i2c.write(LSM6DSL_ADDR, &r, 1, true) != 0) return false;
    if (i2c.read(LSM6DSL_ADDR, &r, 1) != 0) return false;
    val = (uint8_t)r;
    return true;
}

bool read_int16(uint8_t reg_low, int16_t &val) {
    uint8_t lo, hi;
    if (!read_reg(reg_low, lo)) return false;
    if (!read_reg(reg_low + 1, hi)) return false;
    val = (int16_t)((hi << 8) | lo);
    return true;
}

bool init_sensor() {
    uint8_t who;
    if (!read_reg(WHO_AM_I, who) || who != 0x6A) {
        printf("Sensor not found\r\n");
        return false;
    }

    write_reg(CTRL3_C, 0x44);
    write_reg(CTRL1_XL, 0x40);           // 104Hz, ±2g
    write_reg(CTRL2_G, 0x40);           // 104Hz, ±250 dps
    write_reg(INT1_CTRL, 0x03);
    write_reg(DRDY_PULSE_CFG, 0x80);

    ThisThread::sleep_for(100ms);

    uint8_t dummy;
    read_reg(STATUS_REG, dummy);

    int16_t temp;
    for (int i = 0; i < 6; i++) {
        read_int16(OUTX_L_XL + i * 2, temp);
    }
}

```

```
}

int1_pin.rise(&data_ready_isr);
return true;
}

// High priority thread  reads sensor as soon as data is ready
void acquisition_task() {
    while (true) {
        if (data_ready) {
            data_ready = false;

            int16_t acc[3], gyro[3];
            for (int i = 0; i < 3; i++) {
                read_int16(OUTX_L_XL + i * 2, acc[i]);
                read_int16(OUTX_L_G + i * 2, gyro[i]);
            }

            // ±2g: 0.061 mg/LSB
            latest.acc[0] = acc[0] * 0.000061f;
            latest.acc[1] = acc[1] * 0.000061f;
            latest.acc[2] = acc[2] * 0.000061f;

            // ±250 dps: 8.75 mdps/LSB
            latest.gyro[0] = gyro[0] * 0.00875f;
            latest.gyro[1] = gyro[1] * 0.00875f;
            latest.gyro[2] = gyro[2] * 0.00875f;
        }
        ThisThread::sleep_for(1ms);
    }
}

// Normal priority thread  prints at a comfortable rate
void print_task() {
    while (true) {
        printf(">acc_x:%.3f\n>acc_y:%.3f\n>acc_z:%.3f\n"
            ">gyro_x:%.2f\n>gyro_y:%.2f\n>gyro_z:%.2f\n",
            latest.acc[0], latest.acc[1], latest.acc[2],
            latest.gyro[0], latest.gyro[1], latest.gyro[2]);

        ThisThread::sleep_for(100ms);
    }
}

int main() {
    static BufferedSerial pc(USBTX, USBRX, 115200);
    i2c.frequency(400000);

    if (!init_sensor()) {
        while (true) { ThisThread::sleep_for(1s); }
    }

    Thread acq_thread(osPriorityHigh);
    Thread print_thread(osPriorityNormal);
```

```
    acq_thread.start(acquisition_task);  
    print_thread.start(print_task);  
  
    while (true) {  
        ThisThread::sleep_for(1s);  
    }  
}
```

What is Happening Here

The acquisition thread runs at high priority. Every 1ms it checks the `data_ready` flag. When the sensor fires its interrupt and sets the flag, the acquisition thread picks it up, clears the flag, and reads the six axes over I2C. Because it has higher priority than the print thread, if the print thread happens to be running at that moment, it gets preempted immediately and the read happens without delay.

The print thread runs at normal priority. It wakes up every 100ms, grabs the latest values from the shared struct, and prints them. It does not need to run at 104Hz because we are not trying to print every single sample we just want a reasonable live view of the data.

The main thread does nothing useful after setup. It just sleeps to keep the program alive.

What Happens if You Get the Priorities Wrong

Try flipping the priorities: give the print thread high priority and the acquisition thread normal priority. What you will see is that `printf` can take long enough that the acquisition thread gets starved. The sensor keeps firing its interrupt, the flag gets set, but the acquisition thread cannot get CPU time because the print thread outranks it. You will start missing samples.

This is the practical argument for thinking carefully about priorities. The RTOS gives you the tool, but you have to use it correctly.

Single Thread vs Multiple Threads Comparison

To tie everything together, here is how the three approaches we have covered across recitations compare:

Single thread (polling) one loop does everything in sequence. Simple to write and easy to reason about, but tasks interfere with each other's timing. You cannot give two tasks truly independent rates. Fine for very simple programs.

Single thread with interrupts interrupts handle time-critical events immediately, background loop handles less urgent work. Works well for simple cases but gets hard to manage as complexity grows. Everything is still coupled through shared global state.

RTOS threads each task gets its own thread with its own stack and its own rate. The RTOS scheduler handles switching. Tasks are cleanly separated. Scales well as you add more tasks. This is the right model for any embedded system with more than one or two things happening at once.

A Few Things to Keep in Mind

Stack size each thread gets its own stack. By default Mbed allocates 4KB per thread. If your thread uses a lot of local variables or calls deep chains of functions, you may need to increase this. You pass the stack size as the second argument to the Thread constructor:

```
Thread t(osPriorityNormal, 8192); // 8KB stack
```

Thread functions should not return if a thread function returns, the thread terminates. Usually thread functions contain an infinite `while(true)` loop. If you do need a thread to terminate gracefully, call `ThisThread::exit()`.

volatile still matters any variable shared between a thread and an ISR must be declared `volatile`. The compiler does not know that the ISR can modify the variable asynchronously, so without `volatile` it may cache the value in a register and never see updates. We covered this last recitation and it still applies here.

Shared data between threads in this recitation we are reading from `latest` in the print thread while the acquisition thread writes to it. For this demo it works acceptably because the writes are small and atomic on a 32-bit processor. Next recitation we will cover mutexes, which are the proper way to protect shared data when you cannot make that assumption.

References

- Recitation 6 Accelerometer and Gyroscope (LSM6DSL, I2C, Interrupts)
- <https://os.mbed.com/docs/mbed-os/v6.16/apis/scheduling-rtos-and-event-handling.html>
- <https://os.mbed.com/docs/mbed-os/v6.16/apis/scheduling-concepts.html>
- <https://os.mbed.com/docs/mbed-os/v6.16/apis/scheduling-options-and-config.html>
- <https://os.mbed.com/docs/mbed-os/v6.16/apis/scheduling-tutorials.html>